



Arkitekturbeskrivning

Version 1.0

Hanna Olsson

14 mars 2025

Dokumenthistorik

Version	Datum	Förändring och kommentar	Ansvarig
1.0	2025-03-14	Första utkast	Hanna

Projektidentitet - Medarbetarpulsen

VT25, Grupp 11

Linköpings Tekniska Högskola, IDA

Namn	Ansvar	Telefon	E-post
Johanna Lindquist	Teamledare	073-513 99 33	johli622@student.liu.se
Astrid Pettersson	Analysansvarig	073-849 38 39	astpe354@student.liu.se
Fares Espiro	Konfigurationsansvarig	073-622 82 87	fares535@student.liu.se
Johannes Graf	Dokumentansvarig	073-500 41 00	henpe071@student.liu.se
Jonathan Elenstål	Testledare	070-518 31 34	jone1916@student.liu.se
Hanna Olsson	Arkitekt	076-146 80 18	hanol504@student.liu.se
Doaa Shaikh Hussein	Kvalitetssamordnare	072-842 04 83	doash196@student.liu.se
Eric Löfgren	Utvecklingsledare	070-616 28 30	erilo496@student.liu.se

Kund: Personalkollen Sverige AB, Klostergatan 12, 582 23 Linköping,
info@personalkollen.se, kundtelefon 010-150 15 00

Kontaktperson hos kund: Jesper Jonsson, jesper@personalkollen.se, 076-131 04 43

Kursansvarig: Lena Buffoni, lena.buffoni@liu.se, 013-28 40 46

Handledare: Emma Strömberg, emmst476@student.liu.se

Innehåll

1	Inledning	1
1.1	Definitioner	1
1.2	Syfte	1
1.3	Mål och filosofi	1
1.4	Systemets livslängd	1
1.5	Utmaningar	2
1.6	Antaganden och beroenden	2
1.6.1	Beroenden	2
1.6.2	Antaganden om kunskap	2
2	Arkitektoniskt signifikanta krav	2
3	Beslut, begränsningar och motiveringar	3
3.1	Begränsningar	3
3.2	Generella systembeslut	3
3.3	Beslut angående användargränssnitt	4
4	Arkitekturmekanismer	4
4.1	Uthållighet	4
4.2	Schemaläggning	4
5	Nyckelabstraktioner	5
5.1	Enkätantering	5
5.2	Analysverktyg	5
5.3	Anonymitet	5
6	Arkitekturmönster	5
7	Arkitekturvyer	5
7.1	Logisk vy	5
7.1.1	Systemanatomi	6
7.1.2	GUI	7
7.1.3	Identifiering och beskrivning av klasser	7
7.1.4	Spara information i databas	14
7.1.5	Anpassade användarklasser	14
7.2	Användarfallsvy	14

1 Inledning

Detta dokument beskriver filosofier, beslut och begränsningar som ligger till grund för systemets design. Dokumentet syftar till att redogöra för de viktigaste arkitektoniska aspekterna såsom motiveringar, viktiga element och tekniska val.

1.1 Definitioner

Definitioner som kommer att användas i dokumentet:

Backend - Avser den delen av systemet som innehåller funktionalitet för att utföra användarfunktioner.

Frontend - Avser den del av systemet som användare kommer interagera med.

Django - Webbramverk för programmeringsspråket Python

HTMX - JavaScript-bibliotek

1.2 Syfte

Syftet med denna **text** är att identifiera, motivera och hantera de aspekter och beslut som gjorts för att utveckla systemet. Detta dokument fungerar som vägledning till projektgruppen **och möjliga** intressenter till projektet. Genom att beskriva och motivera de beslut som fattats för projektets genomförande kan en heltäckande bild göras av systemets arkitektur och de principer som ligger bakom.

1.3 Mål och filosofi

Målet med arkitekturen är att säkerställa att det slutgiltiga verktyget blir skalbart, **lätt att underhålla, vidareutveckla och använda**. För att uppnå detta har arkitekturen utformats med fokus på flexibilitet och långsiktig hållbarhet.

Under verktygets livsperiod kan tekniska förändringar och ökade krav på systemet förväntas, vilket ställer krav på en anpassningsbar arkitektur.

För att möjliggöra framtida förändringar ska arkitekturen ha dessa aspekter i åtanke:

- **Teknikval** - Verktyg som Django och HTMX kan komma att uppdateras med nya versioner. Nya versioner av verktyg är inte alltid bakåtkompatibla, och på grund av detta bör endast stabila versioner av verktyg med långtidsstöd användas.
- **Skalbarhet** - Systemet ska ha en modulär design och löst kopplade komponenter. Verktygets kod ska dessutom vara väldokumenterad med en tillhörande driftmanual.
- **Samtidig användning** - Systemet måste stödja flera samtidiga användare utan konflikt eller dataläckage.

Genom att ha dessa aspekter i åtanke skapas en robust arkitektur som kan anpassas efter framtida behov och tekniska förändringar.

1.4 Systemets livslängd

Systemet förväntas användas av kunden runt sex gånger om året med en begränsad mängd användare. Kortsiktigt bör systemet fungera väl utan eller med minimalt underhåll i upp till fem år. Då **systemet** är byggt i Django, som uppdateras regelbundet, förväntas ett långsiktigt stöd.

För att förlänga systemets livslängd kan ett antal olika potentiella förändringar behöva betraktas. Dessa förändringar samt hur den nuvarande implementeringen kan motverka detta kan ses i tabell 1.

Tabell 1: Möjliga förändringar som påverkar systemets livslängd.

Möjlig förändring	Hantering
Verktyg som Django och HTMX uppdateras och äldre versioner får inte längre support.	Väldokumenterad kod som gör det lätt att migrera till nyare versioner. Alternativt kan den äldre versionen användas i en virtuell miljö vid användningen av systemet.
Antalet användare ökar och systemet behöver hantera högre belastning.	Valet av databasen bör vara en databas som kan hantera en stor mängd användare. Alternativt ska det vara lätt att byta till en bättre anpassad databas i framtiden.

1.5 Utmaningar

I denna sektion kommer de kritiska utmaningar som arkitekturen måste beakta identifieras. I följande lista kan de utmaningar som hittills identifierats ses:

- Systemet ska kunna visa analyser och trender utan att äventyra anonymitet.
- Systemet ska tillåta flera personer att använda systemet samtidigt.
- Systemet ska kunna analysera trender för återkommande frågor.
- Systemet ska kunna skydda personlig data enligt GDPR.

1.6 Antaganden och beroenden

Här beskrivs de faktorer som påverkar arkitekturen, såsom val av tekniker, eventuella beroenden till andra system och vilka kunskaper som krävs för utvecklingen.

1.6.1 Beroenden

Systemets arkitektur är byggd med beroenden till Django, HTMX samt programmeringsspråken CSS, JavaScript och Python. Dessa tekniker har valts för att möjliggöra en webbaserad lösning, där Django fungerar som webbramverk i Python och HTMX används för att utöka HTML-funktionaliteten. Eftersom data kommer att behöva lagras är arkitekturen också beroende av att använda en databas för att hantera och lagra information.

1.6.2 Antaganden om kunskap

Under utvecklingen av systemet görs antagandet att samtliga projektmedlemmar utökar sin kunskap om relevanta verktyg under utvecklingens gång. Detta gäller särskilt för webbutveckling och databashantering, där gruppens kunskapsnivåer varierar.

2 Arkitektoniskt signifikanta krav

I tabell 2 är projektets mest arkitektoniskt signifikanta krav motiverade. Dessa är de krav som kommer att påverka hur arkitekturen utvecklas. Observera att kravnumren motsvarar det kravnummer som kravet har i kravspecifikationen samt att medarbetare används som ett annat ord för enkätsvarare.

Tabell 2: *Signifikanta krav*

Kravnr	Krav	Motivering
3.1.9	Systemet ska kunna hantera en ökning av data och användare utan märkbar prestandaförsämring.	Detta påverkar att systemet ska vara skalbart och kunna hantera flera användare på samma gång.
3.1.12	Generering av analyser ska ske under 5 sekunder .	För att detta krav ska uppfyllas ställs krav på arkitekturen då den behöver ha effektiv datahantering.
3.2.7	Gränssnittet ska fungera på mobil, surfplatta och desktop utan att användarupplevelsen påverkas.	Om användarupplevelsen ska vara oförändrad för olika skärmstorlekar krävs att arkitekturen utvecklas för att passa dessa storlekar.
4.1.3	Enkätskapare ska, likt medarbetare, kunna svara på enkäter.	Arkitekturen behöver behandla fall då enkätskapare har tillgång till samma funktionalitet som enkätsvarare.
4.3.1	Systemet ska kunna samla in svar från medarbetare och lagra dessa på ett säkert sätt.	För att samla in svar på ett säkert sätt behöver arkitekturen utvecklas enligt Djangos databasmodeller.
4.3.2	Systemet ska stödja filtrering av data.	För att systemet ska stödja datafiltrering på ett hållbart vis behöver arkitekturen utvecklas med algoritmer för filtrering i åtanke.
4.3.4	Vid anonyma enkäter ska svaren vara helt anonyma för att upprätthålla sekretess och integritet för medarbetarna.	För att användare ska förbli anonyma måste arkitekturen utvecklas så att den inte sparar användaruppgifter på ett ställe som användare kan komma åt.

3 Beslut, begränsningar och motiveringar

I denna sektion presenteras de beslut och begränsningar som har påverkat arkitekturen, tillsammans med motiveringar för dessa val. Det inkluderar faktorer som val av databashanterare, användning av tredjepartsprogramvara och andra tekniska begränsningar som påverkar systemets design och implementering.

3.1 Begränsningar

De begränsningar som arkitekturen behöver följa är krav som kunden har ställt på utvecklingen av systemet. Dessa krav är att Django och HTMX ska användas i implementeringen av systemet och därmed kommer Django att fungera som backend för systemet. Detta innebär att systemet kommer att utvecklas som en applikation inom ramen för ett Django-projekt, där den etablerade filstrukturen för Django kommer att tillämpas. Dessutom kommer **HTMX att användas som utvecklingsverktyg** för att hantera de interaktiva funktioner som systemet kräver.

3.2 Generella systembeslut

Det har bestämts att systemet kommer att implementera en databas med **SQLite-struktur**, vilket är ett praktiskt verktyg under utvecklingsfasen. Detta är särskilt relevant då utvecklingen kommer att ske parallellt på olika bärbara datorer, var och en med separata instanser av databasen. SQLite uppvisar dessutom många likheter med PostgreSQL och därmed kommer det att vara relativt enkelt att migrera till PostgreSQL i framtiden, om det skulle bli nödvändigt att hantera en större belastning. Detta är relevant eftersom PostgreSQL är ett databassystem som är kapabelt att hantera flera användare och därmed bättre än SQLite i den aspekten.

Utvecklingsteamet har beslutat att systemet ska utvecklas med olika användarroller för att möta behovet av varierande behörigheter bland användarna. De definierade rollerna är administratör, enkätskapare och enkätsvarare. För att möjliggöra för enkätskapare att skicka enkäter till olika organisationsgrupper kommer olika behörighetsnivåer att implementeras. Detta beslut syftar till att skapa en tydlig distinktion mellan användarnas behörigheter till olika funktioner, vilket i sin tur kommer att underlätta utvecklingen av webbapplikationen.

Det har även beslutats att systemet ska köras med både backendlogik och databas på samma server. Detta arrangemang är nödvändigt eftersom produkten ska drivas internt hos beställaren, vilket eliminerar behovet av en extern databas.

Systemet kommer dessutom att inkludera en frågebank för att underlätta analysen av trender för specifika, återkommande frågor.

3.3 Beslut angående användargränssnitt

Gällande gränssnittet ska det vara användarvänligt och ha ett konsekvent utseende. För att följa ett konsekvent utseende ska användargränssnittet följa ett färgschema och implementeras så att alla vyers färger lätt kan ändras om färgschemat ändras. Utöver detta ska det finnas en navigationsmeny för navigering mellan olika gränssnitt, där användarens tilldelade roll påverkar vilka sidor som den har tillgång till.

4 Arkitekturmekanismer

Arkitekturmekanismer är återanvändbara lösningar som används för att hantera viktiga aspekter av systemets design, såsom datalagring, säkerhet och prestanda. Denna sektion beskriver de arkitekturmekanismer som identifierats för Medarbetarpulsen och hur de tillämpas i systemet.

4.1 Uthållighet

Medarbetarpuls är ett enkätssystem och på grund av detta kommer systemet att hantera, bearbeta och spara data om användare, skapandet av enkäter samt enkäternas resultat. Därmed behöver många instanser av objekt vara uthålliga, alltså kunna existera över en varierad längd av tid och ska kunna både läsas och ändras.

De attribut som behöver identifieras för alla klasser vars instanser bör bli uthålliga är:

- **Mängd** - Hur många instanser av objekten behöver vara uthålliga?
- **Uppdateringsfrekvens** - Hur ofta ska objekten ändras?
- **Pålitlighet** - Ska objekten kunna överleva om processer eller systemet kraschar?
- **Livslängd** - Hur länge ska objekten sparas innan de arkiveras eller raderas?
- **Hämtningsmetod** - Hur ska objektet identifieras och hämtas?

Dessa attribut behandlas i avsnitt 7.1.3 för respektive klass.

4.2 Schemaläggning

Flera av Medarbetarpulsens funktioner kräver schemaläggning för att de ska fungera. Dessa är bland annat funktionen att schemalägga utskick av enkäter och påminnelser samt deadline för medarbetarnas svar på en enkät. Funktionen av arkitekturmekanismen schemaläggning är att ge systemet möjligheten att utföra specifika uppgifter vid specificerad tid.

Följande attribut är de som behöver identifieras för de funktioner som behöver kunna schemaläggas:

- **Tidpunkt** - När ska uppgiften utföras? Exempelvis ett exakt klockslag eller med återkommande intervaller.
- **Frekvens** - Hur ofta ska uppgiften köras? Engångshändelse, dagligen, veckovis etc.

- **Varaktighet** - Om uppgiften har en pågående effekt, hur länge ska den vara aktiv?
- **Felhantering** - Vad händer om en schemalagd uppgift misslyckas eller inte kan köras i tid?

5 Nyckelabstraktioner

Nyckelabstraktioner representerar de mest centrala koncepten i systemet och utgör grunden för dess design och funktionalitet. Denna sektion beskriver de viktigaste abstraktionerna, såsom enkäthantering, anonymitet och analysverktyg, vilka tillsammans definierar systemets kärnfunktioner.

5.1 Enkäthantering

En central funktion hos verktyget är hanteringen av enkäter, utskick av enkäter och insamlingen av dess svar. Varje enkät behöver kunna lagra svaren för framtida analys, samtidigt som det ska vara möjligt att skicka ut samma enkät flera gånger och kunna ändra en enkät utan att det ska påverka redan utskickade enkäter. Därmed behöver enkäter som enkätskapare kan redigera och skicka ut lagras separat från de utskickade enkäterna.

5.2 Analysverktyg

För att enkätanalyser ska kunna visualiseras behövs ett verktyg som tar fram rimliga analyser på enkätsvar. Detta kommer att åstadkommas genom en uppsättning **funktioner, eller klassmetoder**, i systemets backend. Dessa kommer initialt att använda aktuella resultat och medeltal tillsammans med äldre sparade värden. Därefter kommer dessa funktioner kunna returnera trendanalyser på de aktuella värdena, baserat på jämförelse med de äldre. För att kunna göra detta behöver svaren för enkäter sparas så att det är lätt att hitta alla svar som är kopplade till återkommande frågor.

5.3 Anonymitet

En annan viktig del av projektet är att säkerställa att de som svarar på utskickade enkäter förblir anonyma för andra användare. Detta innebär att varken en enkätskapare eller administratör, på något vis, ska kunna hitta information som kan avslöja vem som svarat vad. Om administratörer exempelvis har tillgång till delar av databasen innebär detta att systemet måste ha en arkitektur som aktivt döljer användarinformation från dem men som samtidigt ger analysverktygen tillgång till all data som de behöver.

6 Arkitekturmönster

Arkitekturen kommer att följa en klient-serverarkitektur med Django som backend och HTMX som frontend. Detta är eftersom systemet kommer att ha en central databas som sparar all enkäters data och därmed är det lämpligt att ha en central server som hanterar databasen.

7 Arkitekturvyer

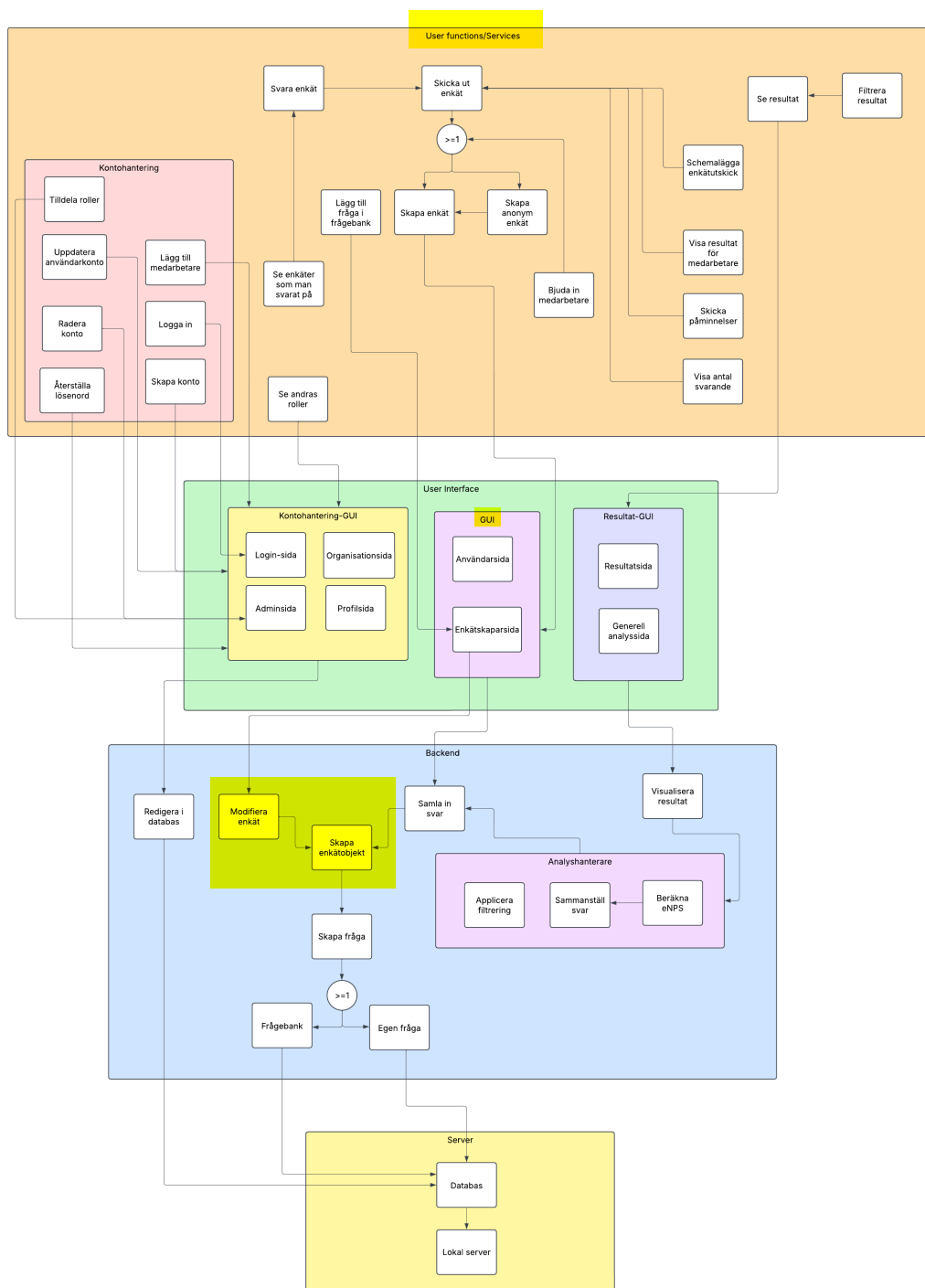
I denna sektion kommer arkitekturen för systemet att beskrivas ur olika perspektiv med störst fokus på den logiska vyn.

7.1 Logisk vy

Systemet kommer att bestå av en webbsida, där ett användargränssnitt kommer att tillåta användaren att använda den funktionalitet som backenden erhåller. På webbsidan kommer användare, beroende på sin roll eller behörighet, **kunna skapa och svara på enkäter**. Dessa enkäter ska vara interaktiva och utformade för att engagera användarna och uppmuntra till svar. För att sedan spara svaren kommer backenden använda en databas där svaren och annan information kan sparas.

7.1.1 Systemanatomi

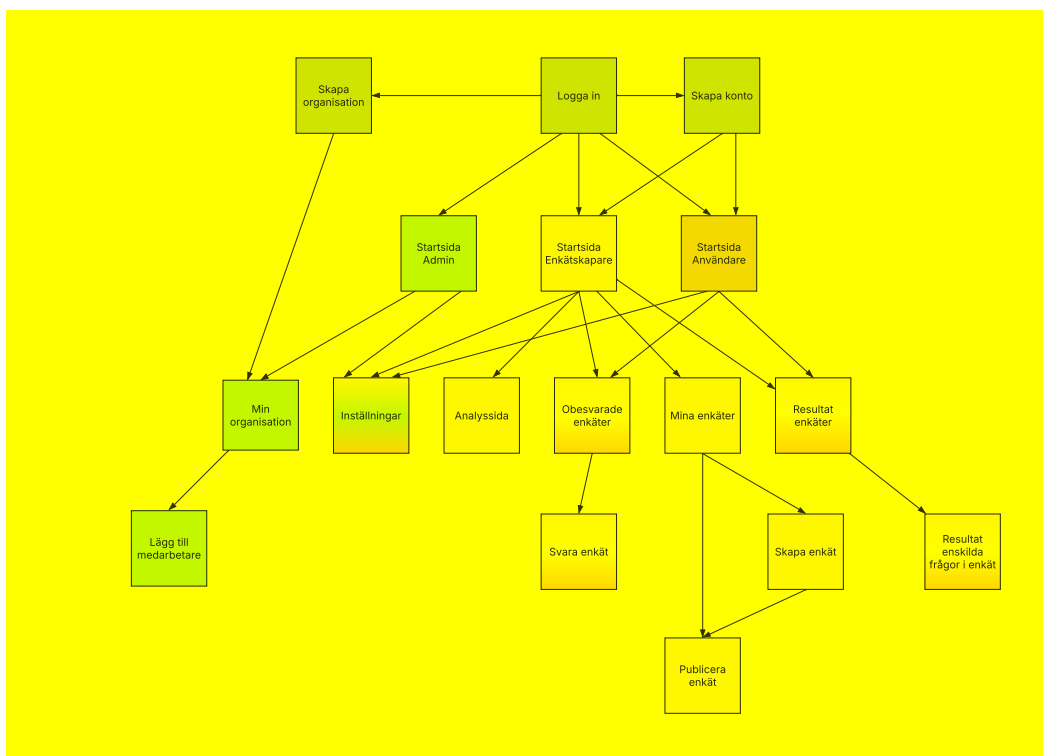
I projektets systemanatomi, se figur 1, är kommunikationen mellan systemets olika delar illustrerad. I de olika större färgade områdena i figur 1 är de olika “nivåerna” som anatomin är indelad i markerade. Här innehåller området “User functions/Services” de olika funktioner som en användare kan förvänta sig få tillgång till när produkten används. En nivå nedanför innehåller området “User Interface” de olika sidor som användaren kommer att interagera med i gränssnittet när de använder en av produktens funktioner. Slutligen innehåller de lägsta nivåerna “Backend” och “Server” de funktioner som systemet kommer att utföra i bakgrunden för att utföra uppgifter och ändra gränssnittet utifrån de uppgifter som kallats på i de högre nivåerna.



Figur 1: Projektets systemanatomi

7.1.2 GUI

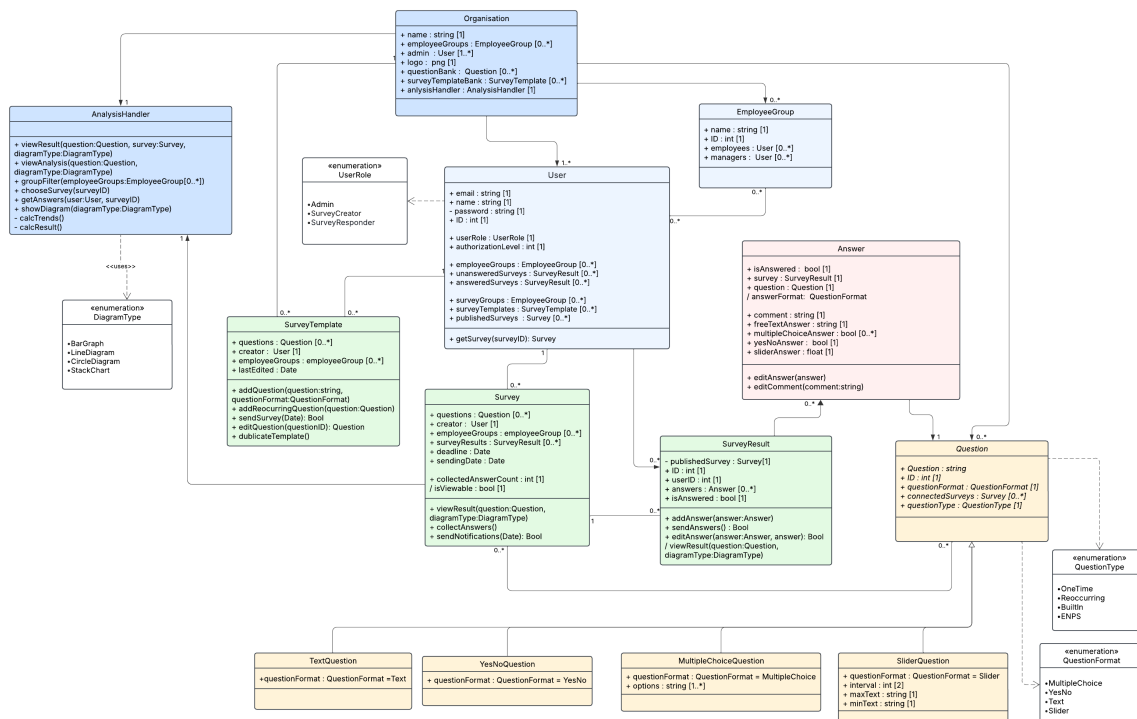
De gränssnitts-vyer som erbjuds i verktygets GUI, samt flödet mellan dem, visualiseras i figur 2. Observera att den blå färgen markerar vyer som är tillgängliga för alla användare, medan gröna vyer är exklusiva för administratörer, gula vyer endast för enkätskapare, och rosa vyer endast för vanliga användare. Blandade färger används för att visa åtkomstnivåer där olika roller delar åtkomst till samma vy. Notera också att samma vy kan ha olika utseende beroende på användarens roll. Exempelvis kan alla användare komma åt inställningssidan, men innehållet och funktionerna på sidan kommer att anpassas beroende på användarens roll.



Figur 2: De olika gränssnitts-vyerna hos verktyget. Blå vyer är för alla användare, gröna för administratörer, gula för enkätskapare och rosa för vanliga användare.

7.1.3 Identifiering och beskrivning av klasser

För att systemet ska kunna utvecklas behöver de olika klasser som behövs för att, dels spara undan relevant information i en databas, och dels för att erhålla en bra kodstruktur, identifieras. I figur 3 är de klasser som kommer att användas i projektet illustrerade i en form av ett klassdiagram, där även samverkan mellan klasserna visas.



Figur 3: Klassdiagram för systemet Medarbetarpuls.

7.1.3.1 Översättning och beskrivning av klasser och fält

Alla klasser, funktioner och fält är enligt praxis på engelska. I tabell 3 är översättningen av klasserna samt översättning och beskrivning av enumerationer och viktiga fält.

Typ	Översättning och beskrivning
Organisation	Organisation
<i>questionBank</i>	Frågebank. Sparar återkommande frågor.
<i>surveyTemplateBank</i>	Enkätmallsbank. Sparar gemensamma enkätmallar.
AnalysisHandler	Analyshanterare. Hanterar analyser av enkäter.
User	Användare
<i>UserRole</i>	Användarroll. De olika användarrollerna är <i>Admin</i> , <i>SurveyCreator</i> och <i>SurveyResponder</i> som står för administratör, enkätskapare respektive enkätsvarare.
<i>authorizationLevel</i>	Behörighetsnivå
<i>employeeGroups</i>	Lista över medarbetsgrupper som en användare tillhör.
<i>unansweredSurveys</i>	Lista över enkätsvararens obesvarade enkäter.
<i>answeredSurveys</i>	Besvarade enkäter. Enkätsvarares tidigare besvarade enkäter.
<i>surveyGroups</i>	Lista över medarbetargrupperna som en enkätskapare har ansvar för.
<i>surveyTemplates</i>	Enkätmallar. Enkätskaparens redigerbara enkäter som kan skickas ut till enkätsvararna.
<i>publishedSurveys</i>	Publicerade enkäter. Lista över enkätskapares utskickade enkäter.
EmployeeGroup	Medarbetargrupp
<i>managers</i>	Lista över enkätskapare som är ansvarig för en medarbetsgrupp.
SurveyTemplate	Enkätmall. Enkätskapare använder dessa för att redigera och skapa enkäter.
Survey	Enkät. Mer specifikt en publicerad, det vill säga utskickad, enkät.
SurveyResult	Enkätresultat för enskilda enkätsvarare.
Question	Fråga
<i>QuestionFormat</i>	Frågeformat. Formatet på en fråga kan till exempel vara att frågan är en flervalsfråga.
<i>QuestionType</i>	Frågetyp. En fråga kan vara en engångsfråga, en återkommande fråga, en inbyggd återkommande fråga eller en ENPS-fråga.
<i>connectedSurveys</i>	Lista över alla publicerade enkäter som är kopplad till en återkommande fråga.
Answer	Svar, det vill säga en enkätsvarares svar på en individuell fråga.

Tabell 3: Översättning och beskrivning av klasser och fält

7.1.3.2 Organisation

Organisation-klassen är grundpelaren i projektet och representerar organisationen. Denna klass ska spara all nödvändig data för organisationen och analyshanteraren samt all data som ska vara gemensam för alla enkätskapare, såsom frågebanken och gemensamma enkätmallar.

Följande är svaren på frågorna i avsnitt 4.1:

- **Mängd** - Det ska enbart finnas ett objekt för organisationen.

- **Uppdateringsfrekvens** - Objektets fält kommer att regelbundet behöva ändras.
- **Pålitlighet** - Objektet behöver kunna överleva krascher av system och processer.
- **Livslängd** - Objektet ska aldrig raderas.
- **Hämtningsmetod** - *Organisation* ska enbart kunna nås från *Admin*-konton och *SurveyTemplate* som ska ha tillgång till frågebanken och enkätmallsbanken.

7.1.3.3 User

User-klassen är klassen som hanterar alla användare. *UserRole* används för att urskilja om en användare är en enkätsvarare (*SurveyResponder*), enkätskapare (*SurveyCreator*) eller administratör (*Admin*) och kommer att användas för att avgöra vilka funktioner och funktionalitet som användaren ska ha tillgång till. Om en användare är en enkätskapare ska den ha tillgång till samma funktionalitet som en enkätsvarare samt ytterligare funktionalitet för skapandet av enkäter.

Det funderades på att ha en abstrakt klass *User* och sedan ha separata klasser för de olika användarrollerna, men det bedömdes att det skulle vara lättast att använda Djangos funktionalitet genom att ha enumerationer för användarrollerna istället. Se avsnitt 7.1.5 för hur Djangos användarklasser ska användas.

Eftersom *User*-klassen hanterar varje användarroll behöver den ha alla fält som de separata användarrollerna behöver ha. Därmed behöver det finnas fält för två listor med medarbetsgrupper; en för enkätsvararen för de medarbetsgrupper som hen tillhör samt en för enkätskaparen för alla grupper som hen ansvarar för. Förutom detta behövs det också en lista för enkätskaparens enkätmallar (*surveyTemplates*), en för enkätskaparens utskickade enkäter (*publishedSurveys*) samt två för enkätsvararens obesvarade respektive besvarade enkäter (*unansweredSurveys* och *answeredSurveys*). Svaren på obesvarade enkäter kan redigeras av en användare men besvarade enkäter ska ej kunna redigeras.

Det valdes att ha två listor av *SurveyResult*-objekt istället för en för att förenkla filtrering och uppdelning av de obesvarade och besvarade enkäterna.

Följande är svaren på frågorna i avsnitt 4.1:

- **Mängd** - Det ska finnas ett objekt för vardera användare och separata *Admin*-konton för administratörer.
- **Uppdateringsfrekvens** - Objektets fält som behandlar enkäter kommer att regelbundet behöva ändras. Andra fält kommer att oregelbundet kunna ändras av administratörer eller enkätskapare.
- **Pålitlighet** - Objektet behöver kunna överleva krascher av system och processer.
- **Livslängd** - Objektet ska enbart raderas när användarkontot inte längre behövs.
- **Hämtningsmetod** - Objekten ska hämtas från *EmployeeGroup* om det är en enkätsvarare eller enkätskapare, eller via *Organisation* om det är en administratör.

7.1.3.4 EmployeeGroup

EmployeeGroup-klassen representerar medarbetargrupper och varje objekt representerar en separat grupp. Klassen används för att gruppera användarna, det vill säga antingen en enkätsvarare eller en enkätskapare. Klassen används också för att gruppera enkätskaparna som ansvarar för respektive medarbetsgrupp.

Följande är svaren på frågorna i avsnitt 4.1:

- **Mängd** - Det ska bara finnas ett objekt för respektive skapad medarbetsgrupp för att undvika onödiga kopior.
- **Uppdateringsfrekvens** - Objekten ska komma att uppdateras oregelbundet beroende på när enkätskapare och administratörer lägger till nya medarbetare och enkätskapare i klassen.

- **Pålitlighet** - Objektet behöver kunna överleva krascher av system och processer.
- **Livslängd** - Objektet ska enbart raderas när en administratör bestämmer att medarbetsgruppen inte längre behövs.
- **Hämtningsmetod** - Objektet hämtas och identifieras ifrån antingen en av de två listorna i *User*-klassen, eller från *Organisation*.

7.1.3.5 *Survey*-klasser

För att skapa, skicka ut och spara svar för enkäter behöver enkäter kunna behandlas i flera olika fall. Enkätskapare ska kunna skapa, redigera, spara och skicka ut enkäter men ska inte kunna svara på sina egna utskickade enkäter. De utskickade enkäterna ska även vara annorlunda från de redigerbara enkäterna eftersom de utskickade enkäterna inte ska kunna ändras i efterhand. Enkätvarare ska också kunna få den utskickade enkäten och spara dess individuella svar.

Detta kan lösas genom att ha en enda enkätklass, men då skulle det bli mycket funktionalitet i en enstaka klass och många specialfall för funktionaliteten. För att undvika detta och för att göra systemet mer modulärt används det tre klasser för enkäthanteringen. Dessa klasser är *SurveyTemplate*, *Survey* och *SurveyResult*.

SurveyTemplate *SurveyTemplate*-klassen representerar enkätmallar som en enkätskapare kan redigera och skicka ut. Denna klass kommer att användas för att spara utkast av enkäter samt för att lätt kunna skicka ut samma enkät flera gånger. Det ska även finnas möjlighet att duplicera ett *SurveyTemplate*-objekt antingen från enkätskaparens lista för enkätmallar eller från den gemensamma enkätmallbanken i *Organisation*.

Följande är svaren på frågorna i avsnitt 4.1:

- **Mängd** - Varje enkätskapare ska ha separata *SurveyTemplate*-objekt i listan *surveyTemplates*. *Organisation*-objektet ska också ha separata *SurveyTemplate*-objekt i listan *surveyTemplateBank*.
- **Uppdateringsfrekvens** - Objektet ska uppdateras när en enkätskapare redigerar och sparar ändringar i enkätmallen.
- **Pålitlighet** - Osparade objekt kan försvinna när systemet och processer kraschar, men sparade objekt ska kunna överleva.
- **Livslängd** - Objekt ska raderas när en enkätskapare vill ta bort en enkätmall. När en *User*-objekt raderas ska även alla *SurveyTemplate*-objekt i *User*-objekts lista *surveyTemplates* raderas. En administratör ska kunna radera enkätmallar från den gemensamma enkätbanken.
- **Hämtningsmetod** - Objektet hämtas och identifieras från *User*-objekts lista *surveyTemplates* eller i *Organisation*-objektets lista *surveyTemplateBank*.

Survey *Survey*-klassen representerar enkätskaparens utskickade enkäter. Denna klass används för att spara all data som behövs för en enkät, såsom frågor, skapare, de medarbetsgrupper som har fått enkäten samt deadline för enkäten. Klassen används också för att samla in enkätvararnas svar och visa resultatet efter antingen deadline har passerat eller när en viss mängd svar från enkätvararna har samlats in. Dessa enkäter ska inte kunna redigeras av enkätskaparen.

Följande är svaren på frågorna i avsnitt 4.1:

- **Mängd** - Det ska enbart finnas ett objekt för vardera utskick av en enkät. Om samma enkätmall skickas flera gånger ska det skapas ett separat *Survey*-objekt per utskick.
- **Uppdateringsfrekvens** - Frågorna, skapare, medarbetsgrupper och utskicksdatum ska aldrig uppdateras. *surveyResult*-fältet ska uppdateras när en enkätvarare skickar in ett svar och *deadline*-fältet ska uppdateras om enkätskaparen vill förlänga tiden för enkäten.
- **Pålitlighet** - Objektet ska överleva om systemet och processer kraschar.

- **Livslängd** - Objekt ska enbart kunna raderas av en administratör. Objekt ska arkiveras efter tre år.
- **Hämtningsmetod** - Objekt ska hämtas och identifieras från *User*-objekts lista *publishedSurveys*, de kopplade *SurveyResult*-objekt och *Question*-objekt som är använd i enkäten.

SurveyResult *SurveyResult*-klassen representerar de individuella svaren från en enkätsvarare på en utskickad enkät. Varje objekt har tillgång till *Survey*-objektet som ska samla in enkätsvararens svar. På grund av detta har *SurveyResult* tillgång till frågorna och deadline för enkäten och därmed behöver den inte spara dessa själv.

Klassen används för att spara och samla in enkätsvararens svar på respektive fråga i enkäten. En enkätsvarare ska kunna påbörja att svara på en enkät och redigera dem senare. Enkätsvarare ska däremot inte kunna redigera sina svar efter hen har skickat in sina svar med funktionen *sendAnswers()* eller om deadline har passerat. När *sendAnswers()* kallas ska fältet *isAnswered* sättas till *True* och det kopplade *Survey*-objektet ska notifieras att ett svar har skickats in. *sendAnswers()* ska enbart kunna skicka svaren om alla kopplade *Answer*-objekt har ett svar att skicka, vilket bedöms utifrån *Answer*-objektets fält *isAnswered*.

Följande är svaren på frågorna i avsnitt 4.1:

- **Mängd** - Vardera enkätsvarare ska ha en egen unik *Survey*-objekt för varje enkät som har skickats till dem.
- **Uppdateringsfrekvens** - Objektet ska uppdateras varje gång en enkätsvarare sparar ett svar i enkäten. Däremot ska inget kunna uppdateras efter deadline har passerat eller om *isAnswered*-fältet har värdet *True*.
- **Pålitlighet** - Objekten och dess sparade svar ska överleva om systemet och processer kraschar. Osparade svar behöver däremot inte kunna överleva.
- **Livslängd** - Objekt ska raderas enbart om det kopplade *Survey*-objektet raderas eller om deadline för enkäten passerar och svaren är ej inskickad. Objekten ska arkiveras efter fem år.
- **Hämtningsmetod** - Objekt kan hämtas och identifieras från en av de två listorna för obesvarade och svarade enkäter i *User*-objektet, från *Survey*-objektets lista på dess resultat eller från kopplade *Answer*-objekt.

7.1.3.6 Answer

Answer-klassen representerar individuella svar för specifika frågor. Dessa används för att spara enkätsvararens svar av olika frågetyper samt deras kommentarer på frågor. *Answer*-objekten används av *SurveyResult* för att bedöma om svaren får skickas in via *isAnswered*-fältet. Varje *Answer*-objekt har tillgång till det specifika *Question*-objektet som de är kopplade till. *Answer*-klassen använder sig av enumerationen *QuestionFormat* från *Question*-klassen för att bedöma vilket fält som svaren ska sparas i.

Answer-klassen kunde göras som en abstrakt klass med subklasser liknande *Question*-klassen för vardera frågetyp, det vill säga till exempel ja/nej-frågor och flervälsfrågor. Detta valdes inte för stunden eftersom det enda *Answer* ska göra är att spara svaren till en fråga, och därmed behöver enbart separata fält för de olika typerna av svar. Däremot om *Answer*-klassen skulle behöva ytterligare funktionalitet som är unik för olika frågetyper ska *Answer*-klassen ändras till en abstrakt klass.

Följande är svaren på frågorna i avsnitt 4.1:

- **Mängd** - Det ska finnas ett objekt för respektive fråga i ett *Survey*-objekt.
- **Uppdateringsfrekvens** - Objektet ska uppdateras varje gång en enkätsvarare svarar eller redigerar sitt svar på frågan som är kopplat till objektet. Däremot inget *Answer*-objekt kunna uppdateras efter deadline i *Survey*-objektet har passerat eller om enkätsvararen har skickat iväg sina svar på enkäten.
- **Pålitlighet** - Sparade svar ska överleva om systemet och processer kraschar. Osparade ändringar eller svar behöver däremot inte kunna överleva.

- **Livslängd** - Objekt ska raderas enbart om det kopplade *SurveyResult*-objektet raderas eller om deadline för enkäten passerar och svaren för enkäten är ej inskickade.
- **Hämtningsmetod** - Objekt kan hämtas och identifieras från listan i *SurveyResult*.

7.1.3.7 Abstrakta klassen *Question*

Question-klassen är en abstrakt klass som representerar en fråga som används i enkäter. En fråga kan antingen vara återkommande och användas i flera olika enkäter, eller en engångsfråga som enbart används en gång i en enkät. För att bedöma detta samt ytterligare aspekter används enumerationen *QuestionType*. Dessa visas och beskrivs i följande lista:

- **OneTime** - Visar att frågan är en engångsfråga.
- **Reoccurring** - Visar att det är en återkommande fråga som har skapats av en enkätskapare och kan tas bort från frågebanken. Då blir frågan en engångsfråga i alla utskickade enkäter som den används i.
- **BuiltIn** - Visar att det är en återkommande fråga som inte kan tas bort från frågebanken.
- **ENPS** - Visar att det är en återkommande fråga som används för att beräkna ENPS och som inte kan tas bort från frågebanken.

Återkommande frågor kan analyseras för att visa trender på deras svar. För att göra detta lättare har varje *Question*-objekt en lista över de utskickade enkäterna som de är kopplade till för att lättare hitta alla svar på frågan.

Varje subklass till *Answer* är en viss specifik frågetyp. Dessa subklasser är *MultipleChoiceQuestion*, *YesNoQuestion*, *TextQuestion* och *SliderQuestion*. Subklasserna används för att behandla de skilda funktionaliteterna som behövs för de olika frågetyperna.

Anledningen till att *Question* är en abstrakt klass är för att det ska vara lätt att kunna lägga till nya typer av frågor utan att påverka implementeringen av andra frågetyper.

Följande är svaren på frågorna i avsnitt 4.1:

- **Mängd** - Det ska enbart finnas ett objekt per återkommande fråga. Det ska skapas kopior av engångsfrågor i *Survey*-objekt när en enkät skickas ut.
- **Uppdateringsfrekvens** - Objekt för återkommande frågor ska aldrig uppdateras. Objekt för engångsfrågor kan redigeras i *SurveyTemplate*-objekt men de objekt som är kopplade till ett *Survey*-objekt ska aldrig uppdateras.
- **Pålitlighet** - Alla objekt för återkommande frågor, sparade engångsfrågor i *SurveyTemplate*-objekt eller objekt i *Survey*-objekt ska överleva om systemet och processer kraschar. Osparade engångsfrågor i *SurveyTemplate*-objekt behöver däremot inte kunna överleva.
- **Livslängd** - Alla objekt för återkommande frågor ska aldrig raderas. Objekt kopplade till engångsfrågor kan enbart raderas av en enkätskapare om objekten är kopplade till ett *SurveyTemplate*-objekt. Om en *Survey*-objekt raderas ska alla kopplade engångsfrågor-objekt raderas.
- **Hämtningsmetod** - Objekt kan hämtas och identifieras från listorna i *Survey* och *SurveyTemplate* samt kopplade *Answer*-objekt. Objekt för återkommande frågor kan identifieras och hämtas från frågebanken i *Organisation*.

7.1.3.8 *AnalysisHandler*

AnalysisHandler-klassen representerar analysmodulen i systemet. Dess uppdrag är att visa resultat, visa trender och analys av enkäter, skapa diagram samt filtrera enkäterna som ska analyseras. Analys hanteraren ska använda sig av enumerationen *DiagramType* för att välja vilken typ av diagram som den ska skapa.

Klassen *AnalysisHandler* hanterar enbart funktionalitet för analyseringen och därmed behöver inte denna funktionalitet nödvändigtvis vara i en klass. Om detta märks vara ett mer passande sätt att

implementera funktionaliteten för analyser av enkäter ska *AnalysisHandler*-klassen tas bort eller revideras.

Följande är svaren på frågorna i avsnitt 4.1:

- **Mängd** - Det ska enbart finnas ett objekt av denna klass.
- **Uppdateringsfrekvens** - Objekt ska aldrig uppdateras utan ska enbart användas för dess funktionalitet.
- **Pålitlighet** - Objektet bör överleva om systemet och processer kraschar men om den inte skulle överleva kan ett nytt objekt enkelt skapas utan förlorad data. Pågående analyser får däremot försvinna.
- **Livslängd** - Objektet ska aldrig raderas.
- **Hämtningsmetod** - Objektet kan hämtas från *Organisation* eller nås från *Survey*.

7.1.4 Spara information i databas

För att spara relevant information i en databas kommer **Djangos inbyggda modellhantering** att användas. Detta innebär att de datastrukturer som ska sparas i databasen kan representeras av "vanliga" Python-klasser, vilka Django sedan sparar i databasen. För att hantera de datastrukturer som krävs för detta projekt kommer funktionen **"ForeignKey"** att användas, vilken kommer att länka godtyckligt många instanser av en klass till en annan. Koden som kan ses i figur 4 kan exempelvis användas enligt de ovan beskrivna stegen för att spara undan ett "Organisation"-objekt med tillhörande "Medarbetare"-objekt i databasen.

```
from django.db import models

class Organization(models.Model):
    name = models.CharField(max_length=255)

class Member(models.Model):
    organization = models.ForeignKey(
        Organization, on_delete=models.CASCADE, related_name="members"
    )
    name = models.CharField(max_length=255)
    email = models.EmailField(unique=True)
```

Figur 4: Kod för att spara instanser av klasserna *Organisation* och *Medarbetare* i en databas

När specifika objekt behöver uppdateras eller helt nya värden ska sparas i oanvända fält kan den inbyggda "self.save()" -funktionen användas.

7.1.5 Anpassade användarklasser

Django har i sin funktionalitet inbyggda användarklasser som kan användas i systemet. Dessa användarklasser är däremot inte anpassade för projektets system och därmed ska dessa användarklasser överskrivas med våra egna anpassade användarklasser för att kunna bibehålla Djangos funktionalitet samt för att lättare kunna tilldela rollerna enkätsvarare, enkätskapare och administratörer till användarna.

7.2 Användarfallsvy

I systemanatomin, se figur 1, finns funktionaliteten som systemets användare ska ha tillgång till i området "User functions/Services". Användare är uppdelade i roller som representerar de olika aktörerna som finns i systemet; enkätskapare, administratörer och enkätsvarare som är vanliga användare. Bilden för systemets GUI, se figur 2, visar även vilka aktörer som har tillgång till de olika vyerna i användargränssnittet.

Dessa figurer ger en bild för hur användarfallen för användarna ser ut, men de främsta användarfallen är följande:

- Logga in: Alla aktörer ska kunna logga in till deras konton.
- Skapa konto: Alla användare ska kunna skapa ett konto.
- Tilldela roller: Administratörer ska kunna tilldela roller som enkätskapare till andra användare.
- Se organisation: Administratörer ska kunna se användarna och dess roller och medarbetsgrupper.
- Skapa enkät: Enkätskapare ska kunna skapa och redigera enkäter.
- Skicka ut enkät: Enkätskapare ska kunna skicka skapade enkäter till enkätsvarare.
- Svara på enkät: Enkätsvarare och enkätskapare ska kunna svara på enkäter som har skickats till dem.
- Se resultat: Enkätskapare och enkätsvarare ska kunna se resultatet för enkäterna som de har skickat ut eller har svarat på.
- Se analys: Enkätskapare ska kunna se analys och trender utifrån enkätskaparens utskickade enkäter samt kunna filtrera vad som ska analyseras.